Security Documentation

Etel Ride Platform

Version 1.2 | Etel Group

INSA Cyber-Security Audit - Security Findings (Etel Ride scope). Prepared for Etel Group's cyber-security audit application to INSA, referenced in the permission request letter to INSA's Director General, submitted in support of a Road Transport license application with the Ministry of Transport and Logistics.

> Version 1.1 adds Section 10 (security-relevant technology stack, key components and controls, system architecture diagram, data flow diagrams and data flow explanation with trust boundaries), supplied as PDF. Sections 1 to 9 are the findings of version 1.0 and are unchanged. **v1.2 correction:** the Section 10.5.2 walk-through now describes the arrived and passenger-confirmation steps as implemented in RideController.php. Sections 1 to 9 are unchanged.

Scope note

Etel Ride (delivery/, the Ride and Driver apps and their API surface) shares one backend, one dispatch entrypoint (api/boot.php), and one database with Etel Group's other service line on this platform, Etel Delivery. An internal code-level security review was conducted across the full shared codebase (authorization, SQL injection, XSS, session/cookie security, error handling, input validation). This document extracts and presents only the findings relevant to Etel Ride's scope: RideController.php, AuthController.php (shared authentication), ride-relevant AgentController.php/NfcController.php actions, and driver-facing front-end code (driver.js, ride.js). Findings specific to Etel Delivery's order/service-giver functionality are out of scope. Every claim below is traceable to a specific file, line, or function name, produced by reading the actual code, not inferred.

1. Authorization enforcement

Checked: every action in RideController.php and AuthController.php for whether the caller's role and, where relevant, the caller's ownership of the specific ride/resource being read or written is verified server-side.

Fixed

RideController.php had no require_driver()-equivalent anywhere: the only gate on driver-privileged actions was the file-level require_auth($pdo) (any authenticated user, any role). A plain customer-role account could call driver_go_online, driver_get_nearby_requests, driver_accept_ride, driver_hail_customer, driver_start_ride, driver_arrived, driver_complete_ride, driver_cashout, driver_topup_etel_money, etc., accepting and running a real passenger's ride end-to-end and collecting real money, with no check that ride_drivers.onboarded = 1 (vehicle info, selfie, license photo, EtelID identity cross-check all confirmed).

Severity: High (fraud/safety: an unverified account could drive real passengers and handle real money).

Fix applied: a new rides_require_onboarded($pdo, $uid) helper (fails 403 unless ride_drivers.onboarded = 1) now guards every driver-money-moving/ride-accepting action listed above. A plain customer-role account (or a driver mid-onboarding) can no longer accept or run a real ride or move driver-side money. driver_update_profile (the onboarding submission endpoint itself) is deliberately NOT gated, since it is how onboarding begins.

Verified clean

- register (AuthController.php) hard-restricts self-registration to customer/courier roles only; a client cannot self-register as driver or admin. Drivers are promoted server-side only after onboarding approval.
- admin_etelid_login never trusts a client-supplied role.
- Every ride-scoped RideController.php action verifies driver_id/passenger_id ownership (or, for shared actions like send_ride_message, computes the caller's role from whichever column matches and fails 403 if neither does).
- Admin ride/driver-management actions require both the admin role and the relevant capability (e.g. pricing.manage) via role_permissions.

2. SQL injection review

Checked: every $pdo->prepare()/query()/exec() call in RideController.php (183 call sites) and AuthController.php (19 call sites), traced from request input through to where it lands in the SQL string.

Findings: none. No SQL injection found. No files required a fix.

All queries are parameterized (PDO::ATTR_EMULATE_PREPARES => false). Any dynamic column/table name traces to a hardcoded whitelist, never request input; LIMIT/date-range values are always intval()'d or ternary-selected from fixed fragments.

3. XSS / output escaping

Checked: innerHTML/similar sinks in driver.js and ride.js rendering user-controlled fields (ride-history detail, chat messages, driver/passenger names).

Fixed

2 stored-XSS sinks found and fixed (High severity, driver- and passenger-facing): driver.js and ride.js ride-history detail modals rendered fields without escaping. Fixed by reusing the codebase's existing escapeHtml() helper (js/core.js).

Additionally, an escUrl() helper was added to core.js (rejects javascript: / vbscript: / data:text/html schemes) and applied everywhere driver.js/ride.js place a raw image/photo URL into a src="..." attribute (e.g. driver selfie/license thumbnails in admin review screens).

Deploy-path note: the Ride and Driver apps ship as versioned minified bundles (js/releases/ride/{version}/, js/releases/driver/{version}/); these fixes were rebuilt and synced into the live release folders, not left as source-only changes.

Judgment call (resolved)

js/releases/driver/v5/ and v6/ were found missing several expected files (driver-bundle-extra.js, driver-rate-rider.js, driver-announcements-v2.js, sos-button.js) present in v1-v4. If the Driver app's stable/canary channel were pointed at v5 or v6, drivers on that version would 404 on these four files and lose rating, in-app announcements, the SOS button, and premium/nav features. **Status: Fixed**, restored by copying the v4 copies (byte-identical content) into both v5 and v6.

4. Session/cookie security

Checked: RideController.php/AuthController.php's use of boot.php's gen_token()/verify_token()/get_auth_user()/require_auth()/get_bearer()/etel_bump_token_version(), the shared bearer-token scheme used by every Ride action.

Verified clean

- Token lifetime: hard 7-day cap, hash_equals() signature check, token_version mismatch check, no silent refresh, real server-side logout revocation.
- Zero raw setcookie() calls anywhere in delivery/. The one cookie-adjacent code path (etelid_auth.php, bridging to EtelID's SSO, used by Ride's etelid_login) already sets Secure/HttpOnly/SameSite=Lax + 8hr inactivity timeout.

5. Error handling and logging sweep (Ride-relevant files)

Checked: AuthController.php's change_password (shared by all roles including drivers/passengers) and RideController.php for empty/comment-only catch blocks guarding non-trivial operations.

Fixed

AuthController.php's change_password token-refetch failure was found silently soft-degrading (returning a possibly-stale token instead of failing). **Fixed:** it now hard-fails with {success:true, token:null, require_relogin:true}, and both admin and passenger/driver-facing frontends check require_relogin and force an explicit re-login rather than silently continuing with a token that might not work.

6. Input validation audit (Ride-relevant controllers)

Checked: RideController.php and AuthController.php for how incoming $_POST/$_GET/JSON-body values are validated before reaching SQL or a filesystem operation, including the driver document/selfie upload path.

Findings: the general pattern is strong and consistent; no fix required. Zero raw request values reach SQL unparameterized. Numeric IDs are consistently intval()'d; validate.php's shared helpers (etel_validate_phone(), etel_validate_amount()) cover phone and monetary-amount validation used throughout ride booking/fare/payout flows. The driver document/selfie upload path (upload.php) is content-sniffed against a MIME whitelist, size-capped, and stores files under server-generated filenames; no client filename is ever used, preventing path traversal.

7. Machine-to-machine surfaces (Ride-relevant)

AgentController.php (ride actions: agent_ride_quote, agent_book_ride, agent_ride_status, agent_cancel_ride): every action requires the shared service key (constant-time hash_equals() against ETELID_SERVICE_KEY); a customer bearer token alone cannot reach any agent_* action. Member-specific actions additionally resolve a real etel_sso_token to a specific local user, so the service key alone cannot act on an arbitrary member's data.

NfcController.php::nfc_start_ride: gated only by the shared service key with no independent role check on the caller, a deliberate design choice mirroring AgentController.php's pattern and trusting that EtelID has already verified the scanning party's role before calling in. Independently verified: this action does re-check the resolved driver's own onboarding status (ride_drivers.onboarded = 1) server-side before allowing the ride to start, so the NFC path cannot be used to route around the onboarding gate described in Section 1.

Findings: none requiring a fix.

8. Known open items - status (shared infrastructure, Ride-relevant)

8.1 backups/ directory web-accessibility

Found: delivery/api/updater.php creates runtime source-file backups in delivery/api/backups/; the covering .htaccess only set Options -Indexes, with no explicit deny rule and no .htaccess inside the directory itself. A direct request to a backed-up file (which could include shared code Ride depends on, such as config.js) was not explicitly denied.

Fixed: created delivery/api/backups/.htaccess with Require all denied (+ older-Apache fallback).

8.2 Hardcoded admin password in Updater tool

Checked and found already fixed in a prior session: updater.php / updater-api.php load credentials from .env/getenv() and fail closed (503) if unset; no default password, no silent bypass.

9. Sample request/response pairs (Ride-relevant)

See the companion TDD/LLD document, Section 02 (API Contracts), for worked examples of login, book_ride, and driver_topup_etel_money with exact field names taken directly from the implementing code.

Summary of code changes (Ride-relevant files)

RideController.php, AuthController.php, boot.php, js/core.js, js/driver.js, js/ride.js, the corresponding js/releases/driver/{version}/ and js/releases/ride/{version}/ bundles, api/backups/.htaccess (new). Every PHP edit verified with php -l and a non-ASCII-byte scan; every JS edit verified with node --check.

10. Security Architecture, Technology Stack and Data Flow

This section was added in version 1.1 at the INSA reviewer's request. It places the findings in Sections 1 to 9 in the context of the system's technology stack, components, architecture and data flows, and shows which control protects each flow.

10.1 Technology stack (security view)

Layer	Technology	Security-relevant property
Transport	HTTPS to ethelmarket.com (shared cPanel host); JSON over POST to a single endpoint (api/boot.php)	Bearer token in the Authorization header only; no credentials in URLs for user traffic
Server	PHP 8.1, controller-dispatch, no third-party framework	Single gateway applies authentication, rate limiting and action whitelist before any controller runs
Database	MySQL utf8mb4 via PDO prepared statements (emulated prepares disabled)	Eliminates SQL injection (Section 2); dynamic identifiers come from fixed whitelists
Passwords	password_hash() / password_verify()	Passwords stored only as salted hashes
Session	HMAC-SHA256 signed bearer token (user id, issue time, token_version); 7-day cap	Server-side revocation by incrementing token_version; no refresh; hash_equals() comparison
Identity	EtelID SSO; EtelID service key for machine-to-machine calls	Constant-time key check; header with query-parameter fallback
Client	Vanilla JavaScript PWA / Android TWA; versioned minified bundles	Output escaped with escapeHtml()/escUrl(); service-worker cache versioning
Uploads	upload.php	MIME sniffing against whitelist, size cap, server-generated file names, whitelisted folders
Real-time	Server-Sent Events	Authenticated and scoped to the caller's own ride (passenger_id ownership check)
Notifications	Web Push (VAPID, RFC 8292); Telegram Bot API	Signed push requests; bot token held server-side
Operations	In-app Updater with backup and patch_log; cPanel cron	Updater credentials from .env and fail closed (503); backups directory denied by .htaccess; cron endpoints protected by secret key

10.2 Key components and their security controls

Component	Security controls	Threat addressed
boot.php gateway	Token / service-key authentication, rate limiting via login_attempts (login: 5 attempts per 10 minutes with 15-minute lockout; registration: 5 per hour per number and 60 per hour per IP), action whitelist	Credential guessing, unauthenticated or unknown-action access
AuthController	Self-registration limited to customer/courier; change_password hard-fails to forced re-login; admin_etelid_login ignores client-supplied role	Privilege escalation, stale-session reuse
RideController	rides_require_onboarded() on every driver-privileged action; passenger_id/driver_id ownership checks; server-side fare calculation	Unverified driver fraud, cross-user data access, fare tampering
AdminController	require_admin($pdo, capability) against role_permissions	Over-privileged staff accounts
AgentController / NfcController	Shared service key via hash_equals(); SSO token resolves the member; nfc_start_ride re-checks driver onboarding	Forged machine-to-machine calls, onboarding bypass via NFC
upload.php	Content-sniffed MIME whitelist, size cap, generated names	Malicious upload, path traversal
Client bundles (ride.js, driver.js)	escapeHtml(), escUrl(); fixes synced into versioned release bundles	Stored XSS, javascript: URLs
EtelPay bridge	etel_pay_verify_tx() server-side check before a pending top-up row; second admin approval before balance moves	Spoofed payment reference
Updater and backups	Env-based credentials failing closed; backups/.htaccess Require all denied; patch_log audit trail	Unauthorised deploys, source disclosure
Cron scripts	Secret key from .env, standalone (do not load boot.php)	Unauthorised triggering of background jobs

10.3 System architecture diagram

Etel Ride has four tiers: untrusted PWA/TWA clients, a PHP application tier with a single gateway, a MySQL and file-storage data tier, and external services. Every request crosses the gateway, which authenticates it before any controller or database access.

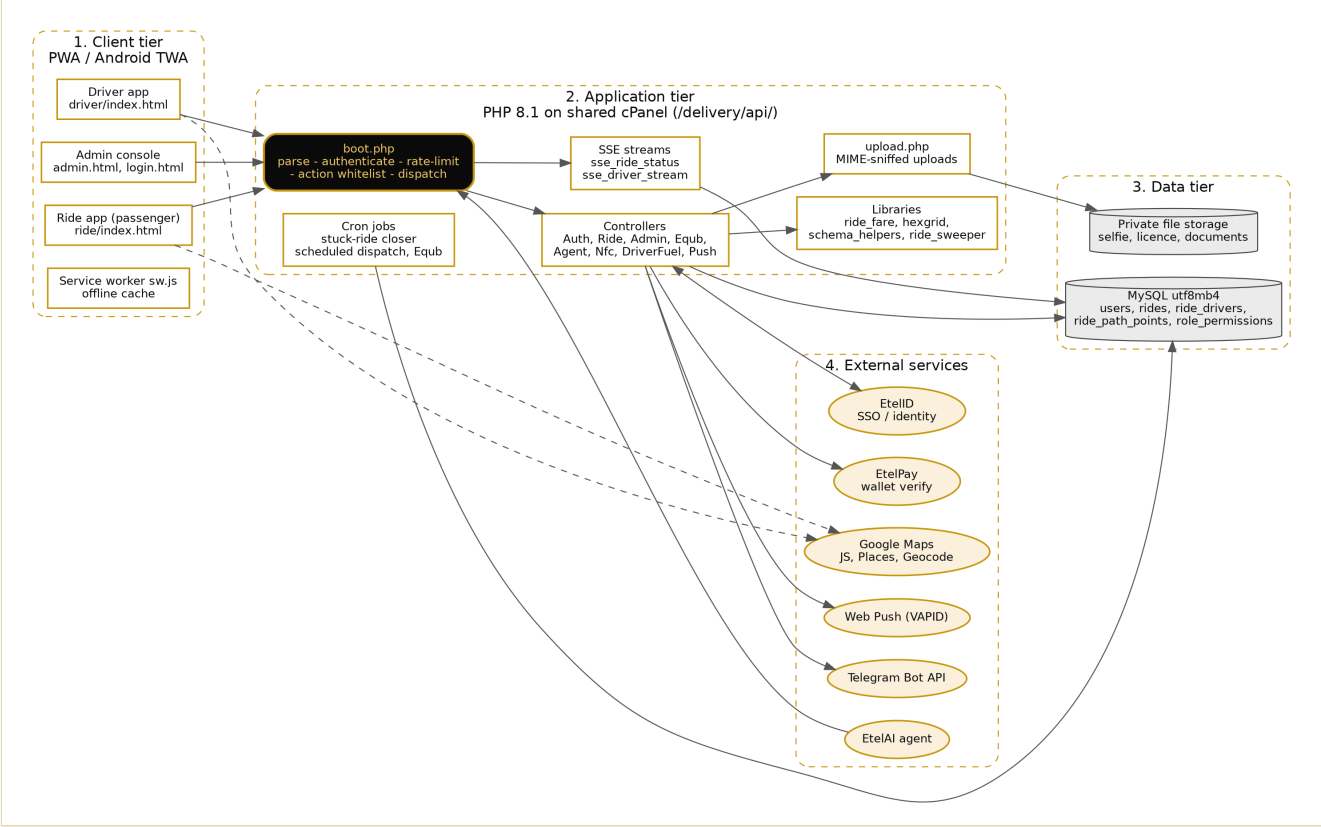


Figure 1 - Etel Ride system architecture

10.4 Data flow diagrams

10.4.1 Level 0 - context diagram

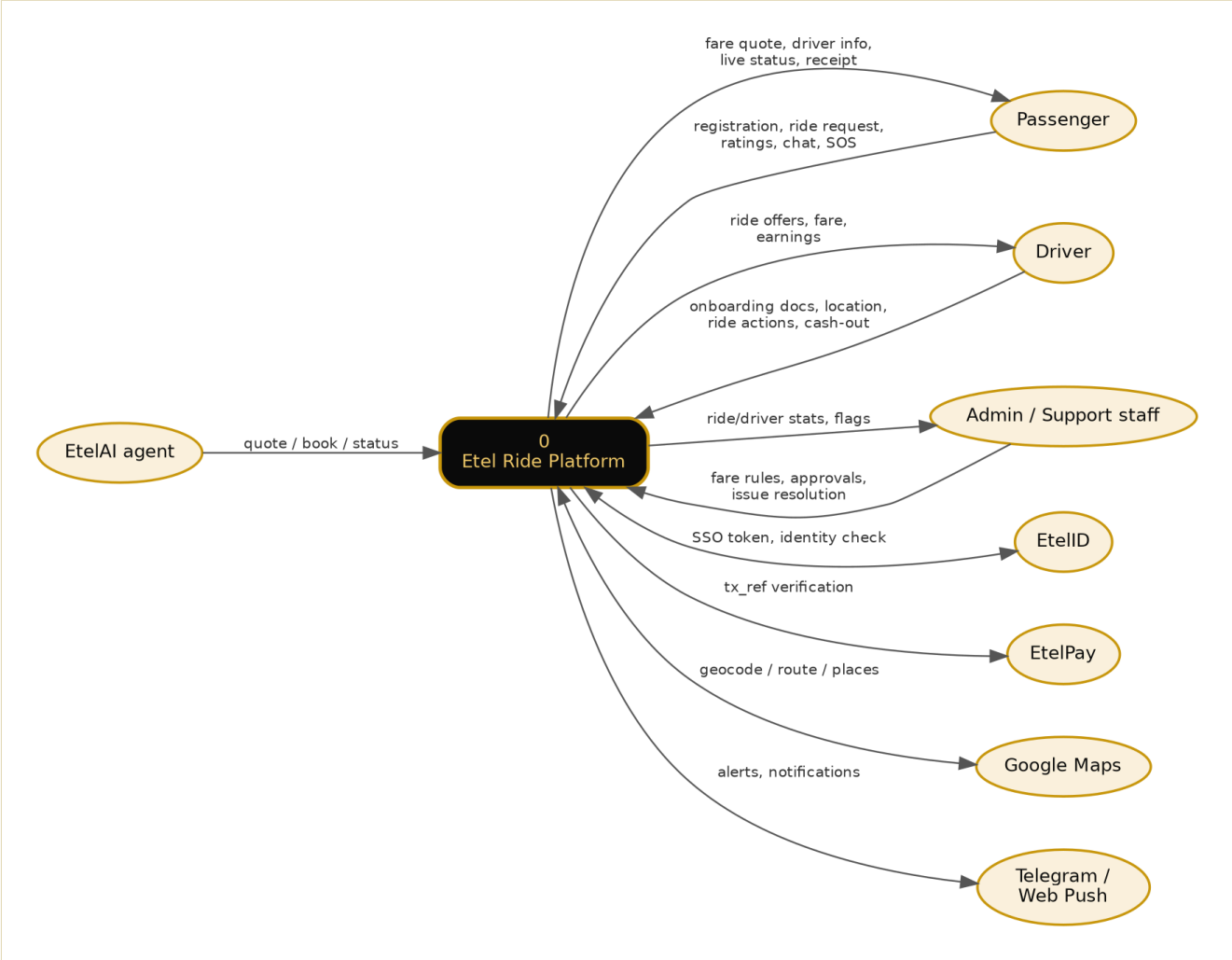


Figure 2 - Level 0 DFD: Etel Ride and its external entities

10.4.2 Level 1 - process decomposition

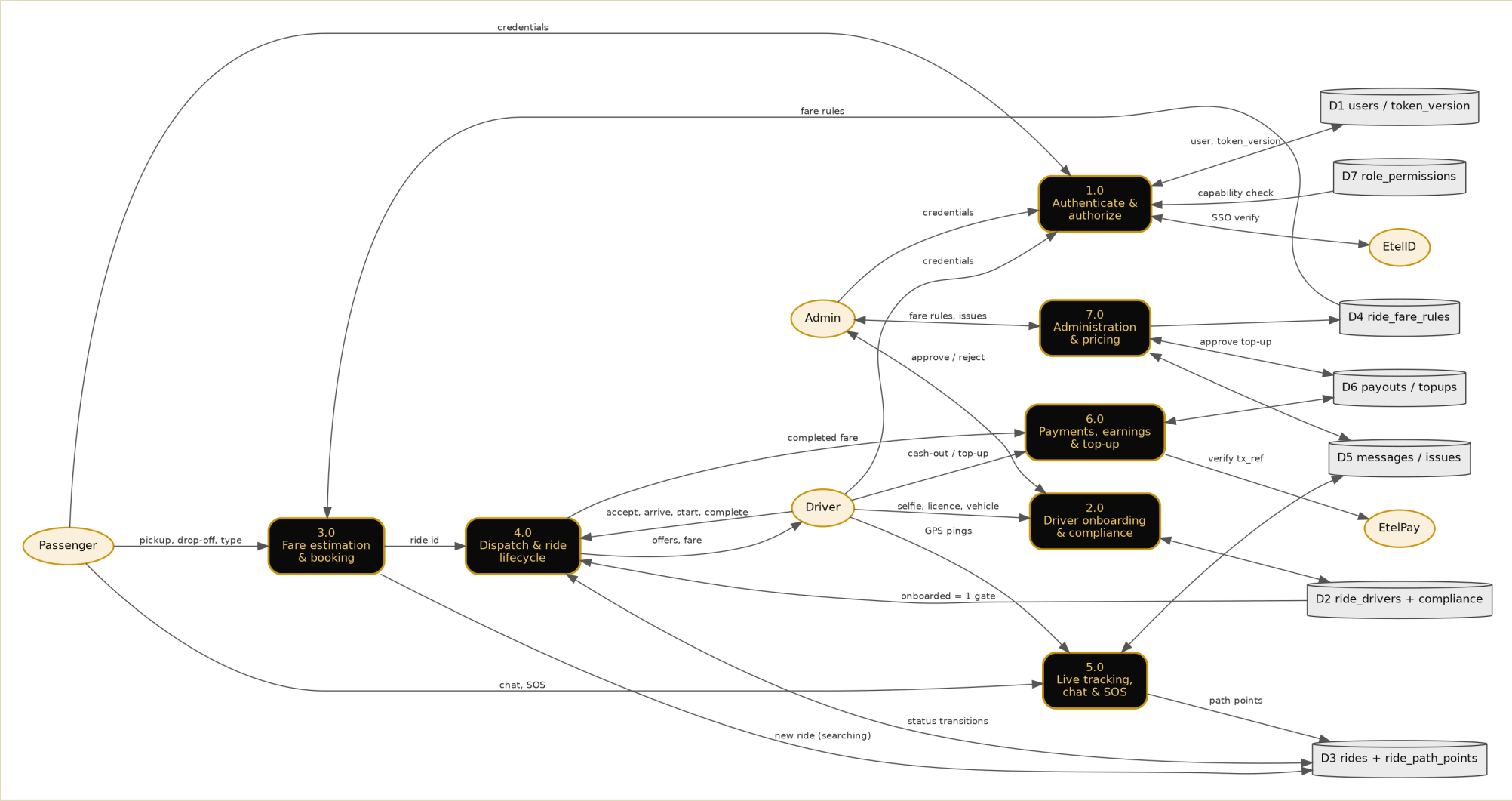


Figure 3 - Level 1 DFD: processes 1.0 to 7.0, data stores D1 to D7, external entities

10.5 Data flow explanation

10.5.1 Trust boundaries and controls per flow

Flow	Trust boundary crossed	Controls applied
Client app to boot.php	Internet to application	HTTPS; bearer token verified (signature, 7-day lifetime, token_version); rate limit on login; action whitelist
boot.php to database	Application to data tier	PDO prepared statements; ownership and capability checks in the controller before any query
Driver upload to file storage	Internet to application to storage	MIME sniffing, size cap, server-generated names; files not served from the client-supplied name
Application to EtelID / EtelPay	Application to group systems	Shared service key over HTTPS, hash_equals(); transaction reference verified server-side
EtelAI / NFC to boot.php	Group systems to application	Service key required; member actions need a resolved SSO token; NFC path re-checks onboarding
Application to Telegram / Web Push	Application to third parties	Tokens and VAPID keys held server-side; payloads limited to alert content
Client browser to Google Maps	Browser to third party	Direct browser call using the Maps JavaScript API; only map rendering and address/place queries are sent

10.5.2 Walk-through: a ride from request to completion

1. **Authenticate (1.0).** The passenger logs in; login is rate-limited and the server issues an HMAC-signed token bound to the user's token_version.
2. **Quote and book (3.0).** The app sends pickup, drop-off and vehicle type. The server reads fare rules (D4), calculates the fare itself and creates a ride in D3 with status searching. Client-supplied fares are never trusted.
3. **Match and accept (4.0).** Nearby drivers are found by hex-grid matching. Accepting requires rides_require_onboarded() (D2); status changes are guarded so a replayed or duplicate action is rejected with a 400.
4. **Track (5.0).** GPS points are stored as ride_path_points. SSE streams ride status and location only to the ride's own passenger and driver. Chat, issues and SOS are stored in D5 and forwarded to staff by Telegram and Web Push.
5. **Complete and pay (6.0).** At the drop-off the driver ends the trip (ride status arrived) and the fare breakdown is stored; the passenger confirms, wallet payments are verified with EtelPay, and the ride is completed through one shared routine that credits earnings and deducts the platform commission. Cash rides left unconfirmed are completed by a timeout sweeper. A driver top-up must pass EtelPay verification and a second admin approval before the balance changes.
6. **Administer (7.0).** Admin actions check the admin role and a named capability in D7; vehicle changes and driver identity flags need admin approval before taking effect.

10.5.3 Data protection summary

- Passwords: salted hashes only. Tokens: not stored; revoked via the per-user token_version counter.
- Documents (selfie, licence): stored under server-generated names; the backups directory is denied at web-server level.
- All SQL parameterized; all user-controlled output escaped before rendering in HTML.

- Service-to-service secrets are compared in constant time and loaded from environment/config, not from request input.
- Updater deploys are logged in patch_log with version, timestamp and operator.